COURSE UNIT: MOBILE COMPUTING

TOPIC: COURSEWORK 1

LECTURER: DR SSEMALULU

GROUP A - MEMBERS

NAME	REG NO.
MUTUME JULIUS	24dsit 0124
ZAWEDDE NASHIBAH	24dsit 0120w
ATUHAIRWE OLIVER	24dsit 0116w

QUESTION 1.

How do you optimize an Android app for performance?

To effectively boost your Android app's performance, you need to target the CPU, memory, and rendering pipeline. Here are 10 proven strategies:

1. Implement Baseline Profiles:

Baseline Profiles allow the Android Runtime (ART) to pre-compile critical code paths during installation. This bypasses the slow "Just-In-Time" (JIT) compilation during the first few launches, improving app startup time by up to 30%.

Source: Android Developers - Baseline Profiles

2. Flatten View Hierarchies with ConstraintLayout:

Deeply nested layouts (like multiple LinearLayouts inside each other) force the system to perform expensive recursive "measure" and "layout" passes. Using ConstraintLayout allows you to create complex UIs with a flat hierarchy, significantly reducing UI thread load.

Source: Android Developers - Build a Responsive UI with ConstraintLayout

3. Use R8 for Code Shrinking and Obfuscation:

The R8 compiler reduces the final size of your APK/AAB by removing unused code and resources. Smaller apps load faster into memory and reduce the "instruction cache" pressure on the CPU.

Source: Android Developers - Shrink, obfuscate, and optimize your app

4. Efficient Image Loading with Glide or Coil:

Loading high-resolution bitmaps directly into memory can trigger OutOfMemory errors or frequent Garbage Collection (GC) pauses. Libraries like Glide or Coil automatically handle downsampling, memory caching, and bitmap pooling to keep the UI smooth.

Source: Glide Documentation - Performance

5. Offload Work via Kotlin Coroutines:

Performing network requests or database disk I/O on the Main Thread causes the UI to freeze (ANRs). Coroutines allow you to switch contexts to Dispatchers.IO or Dispatchers.Default easily, ensuring the UI remains responsive.

Source: Kotlin Documentation - Coroutines on Android

6. Use RecyclerView with ViewHolder Pattern:

For long lists, RecyclerView is essential because it recycles view objects that have scrolled off-screen. This prevents the constant inflation of new views, which is one of the most common causes of "janky" scrolling.

Source: Android Developers - Create dynamic lists with RecyclerView

7. Minimize Overdraw:

Overdraw occurs when the app draws the same pixel multiple times in a single frame (e.g., a background on a layout and another background on the child). Use the "Debug GPU Overdraw" tool to identify and remove unnecessary android:background attributes.

Source: Android Developers - Reduce overdraw

8. Optimize App Startup with Startup Library:

The App Startup library provides a performant way to initialize components at application startup. Instead of defining separate Content Providers for every library (which slows down launch), it merges them into a single provider to reduce overhead.

Source: Android Developers - App Startup

9. Use ViewStubs for Lazy Loading UI:

If your UI has elements that only appear occasionally (like "Error" or "Loading" states), don't include them in the initial layout. Use ViewStub, a lightweight view that consumes zero memory and CPU until you explicitly call inflate().

Source: Android Developers - ViewStub Reference

10. Profile with Macrobenchmark:

You cannot fix what you cannot measure. The Jetpack Macrobenchmark library allows you to write tests that measure large-scale interactions like startup and scrolling. It provides the "truth" on whether your performance tweaks actually worked in production-like environments.

Source: Android Developers - Measurement with Macrobenchmark

References

1. Google Developers: Performance Overview
2. Android Jetpack: Guide to App Performance
3. Kotlin Documentation: Coroutines Guide
4. GitHub / Bumptech: Glide Documentation

QUESTION 2.

What is key when designing a mobile User Interface for small screens?

When designing for small screens, the core challenge is balancing usability with limited space. You must prioritize what the user needs to see and do immediately while minimizing cognitive load.

1. Prioritize Content and Tasks (Visual Hierarchy):

On a small screen, you cannot show everything. Use a clear visual hierarchy to highlight the primary action (e.g., a "Buy" button) while hiding secondary options in menus. This prevents the user from feeling overwhelmed.

Source: Material Design - Layout Hierarchy

2. Design for "The Thumb Zone":

Most users hold their phone with one hand. Key interactive elements (buttons, navigation) should be placed in the bottom or middle of the screen where the thumb can easily reach them without the user shifting their grip.

Source: Smashing Magazine - The Golden Rules of Mobile UX

3. Ensure Adequate Tap Targets:

Small screens make precise clicking difficult. Interactive elements should be at least 48x48 dp (density-independent pixels) with enough space between them to prevent "fat-finger" errors or accidental clicks.

Source: Android Developers - Accessible Tap Targets

4. Minimize Data Entry:

Typing on a mobile device is slow and error-prone. Use features like Autofill, checkboxes, radio buttons, or date pickers instead of free-text fields whenever possible to reduce user friction.

Source: Nielsen Norman Group - Mobile User Experience

5. Use Legible Typography:

Small screens require high-contrast, readable fonts. Aim for a minimum body text size of 16px (or 12-14sp on Android) to ensure users don't have to squint or zoom in to read your content.

Source: Apple Human Interface Guidelines - Typography

6. Optimize for Glanceability:

Mobile users are often distracted. Use icons, bold headings, and clear progress indicators so a user can look at the screen for two seconds and understand exactly where they are and what they need to do.

Source: Interaction Design Foundation - Mobile UI Design

7. Maintain Consistent Navigation:

Avoid complex navigation structures. Use standard patterns like Bottom Navigation bars or Hamburger menus that users already understand, ensuring they don't get lost in "deep" menu trees.

Source: Material Design - Navigation

8. Use Progressive Disclosure:

Instead of crowding the screen with data, show only the essential information first. Use "Read More" links, accordions, or expandable sections to reveal deeper details only when the user requests them.

Source: Nielsen Norman Group - Progressive Disclosure

9. Optimize Images and Whitespace:

Clutter is the enemy of small screens. Use negative space (whitespace) to group related items and give the eye a place to rest. Ensure images are responsive and don't push critical text off the screen.

Source: Adobe XD Ideas - 10 Essential Tips for Mobile UI Design

10. Provide Immediate Feedback:

Because mobile connections can be spotty, users need to know the app is working. Use Skeleton Screens, progress bars, or haptic feedback to confirm that a button was pressed or a page is loading.

Source: Google Developers - Web Fundamentals (Rail Model)

References

1. Google Material Design: Official Layout Foundations
2. Apple Human Interface Guidelines: iOS Design Themes
3. Nielsen Norman Group: Mobile UX Articles
4. Interaction Design Foundation: Principles of Mobile Interface Design

QUESTION 3

How does a mobile app handle offline data?

Handling offline data requires a combination of local storage, synchronization logic, and user experience (UX) patterns to ensure the app remains functional without an active internet connection.

1. Local Database Storage (Room/SQLite):

Mobile apps use a local database to store structured data. By caching API responses in a local SQLite database (often via the Room library on Android), the app can display the last known state of the data immediately when the user opens it offline.

Source: Android Developers - Save data in a local database using Room

2. Implementation of WorkManager for Deferred Tasks:

When a user performs an action offline (like sending a message), the app schedules a task using WorkManager. This ensures the task is "guaranteed" to run as soon as the device regains connectivity, even if the app is closed or the device restarts.

Source: Android Developers - Schedule tasks with WorkManager

3. Repository Pattern for Data Truth:

A Repository acts as a mediator between the local database and the remote API. It implements logic to decide whether to fetch fresh data from the web or serve "stale" data from the local cache based on the connection status.

Source: Android Developers - Guide to app architecture

4. Cache-Aside Pattern (Image Caching):

For visual content, libraries like Glide or Coil use disk caching. They save downloaded images to the phone's internal storage so that if the user scrolls through a list they have previously seen while offline, the images still appear.

Source: Glide Documentation - Caching

5. Optimistic UI Updates:

To make the app feel fast, the UI is updated immediately as if the action succeeded (e.g., a "Like" heart turns red). If the eventual background sync fails after several attempts, the app then rolls back the change and notifies the user.

Source: LogRocket - Optimistic UI updates in mobile apps

6. Conflict Resolution Policies:

When data is changed both offline and on the server, the app must handle conflicts. Common strategies include "Last Write Wins" (using timestamps) or "Manual Merge" (asking the user which version to keep).

Source: AWS Amplify - DataStore Conflict Resolution

7. Delta Sync (Incremental Updates):

Instead of re-downloading the entire dataset when the connection returns, the app requests only the changes (deltas) that occurred since the last successful sync. This saves battery and data plan usage.

Source: Realm (MongoDB) - Synchronized Data Foundations

8. Use of Sync Tokens or Etags:

Apps use Etags or version tokens to check if the server data has changed. If the token matches the local version, the app knows the offline cache is still valid and avoids an unnecessary download.

Source: MDN Web Docs - HTTP ETag

9. Offline-First UI Indicators:

Good offline handling includes a UX layer that informs the user. This involves using subtle banners or status icons (like a "cloud with a slash") to let the user know they are seeing cached data and that their changes are "pending sync."

Source: Material Design - Offline States

10. SharedPreferences or DataStore for Key-Value Pairs:

For small amounts of data, like user settings or simple flags (e.g., "Has finished tutorial"), apps use Jetpack DataStore. This is a non-relational way to keep preferences accessible even without a network.

Source: Android Developers - Store key-value data

References

1. Google Developers: Official Guide to App Architecture
2. MongoDB: Offline-First Mobile App Development
3. Material Design: Communication and Connectivity Guidelines
4. AWS Documentation: Offline Data Access Patterns

QUESTION 4.

Why do we encrypt data in a mobile app?

Mobile apps may contain or deal with sensitive user information such as passwords, financial data, personal messages etc. This data is prone to unauthorized access, hackers, and data breaches which brings about the need to encrypt the data in the mobile app. More of the reasons for encryption are discussed below.

1. Protecting Personal Identification Information (PII):

Encryption ensures that sensitive user data like names, addresses, and social security numbers remain private and inaccessible to hackers.

Source: SoftwareSeni

2. Securing data against Device Theft:

If a device is lost or stolen, encryption prevents unauthorized individuals from reading the local database or cached files.

Source: Kaspersky

3. Preventing Interception on Public Wi-Fi:

Mobile devices often connect to unsecured networks; encryption in transit (TLS/SSL) stops "Man-in-the-Middle" attacks from capturing data as it travels to the server.

Source: FileCloud

4. Meeting Legal Compliance (GDPR/HIPAA):

Encryption is a core requirement for many data protection laws; failing to encrypt can result in massive legal fines and penalties.

Source: ICO.org.uk

5. Ensuring Data Integrity:

Encryption helps verify that the data has not been tampered with or altered by a third party during transmission or storage.

Source: DoveRunner

6. Safeguarding Financial Transactions:

For banking or e-commerce apps, encryption is vital to protect credit card details and transaction histories from financial fraud.

Source: Medium (SilverSky Technology)

7. Maintaining User Trust and Brand Reputation:

Users are more likely to download and use apps that demonstrate a clear commitment to security, whereas a breach of unencrypted data can ruin a company's image.

Source: Impact Techlab

8. Protecting Intellectual Property:

Encryption prevents competitors or malicious actors from reverse-engineering the app to steal proprietary algorithms or business logic stored within the data.

Source: Entrust

9. Safe Harbor from Data Breach Notification Laws:

In many jurisdictions, if encrypted data is stolen, it is not considered a "breach" that requires public notification because the data is unreadable.

Source: AnzaForge

10. Isolating Data in Shared Environments:

For apps using cloud storage or multi-tenant databases, encryption ensures that your data remains isolated and protected even if the underlying server hardware is shared.

Source: DataConversion

References

- Reasons to encrypt your data - Kaspersky
- Top 10 Reasons You Need Encryption - Entrust
- Understanding the Role of Encryption in Protecting Data - Softwareseni
- Encryption and data storage - ICO (UK)
- The Importance of Data Encryption - Dataconversion
- Mobile App Security: Why It Is Important? - Impact Techlab
- The importance of mobile app security - appleute
- The Importance of Encryption on Mobile Devices - Encriptados
- Data At Rest vs In Data Transit - FileCloud
- Encryption at Rest vs. Encryption in Transit - Censinet
- Mobile App Compliance Checklist - AnzaForge

QUESTION 5.

How do you test a mobile app's location -based feature?

Testing a mobile app's location-based features involves a mix of virtual simulation, physical validation, and performance monitoring to ensure accuracy across different regions and environments.

Below are some of the ways to Test Location-Based Features

1. **Mock Location Simulation:**
   Use built-in developer options on Android or tools like Xcode for iOS to "teleport" the device to specific latitude and longitude coordinates. This validates how the app fetches and displays data for any global location without physical travel.

   Source: Pcloudy

2. **Field Testing in Real-World Environments:**
   Physically test the app in diverse settings such as urban centers with tall buildings, suburban areas, and rural zones. This helps identify inaccuracies caused by "GPS multipath" or signal interference from physical obstacles.

   Source: Momentum Suite

3. **Cloud-Based Real Device Testing:**
   Utilize platforms like BrowserStack or LambdaTest to access physical devices hosted in global data centers. This allows you to verify localized content, such as regional pricing and language, on actual hardware in those countries.

   Source: BrowserStack

4. **Geofencing Trigger Validation:**
   Create virtual boundaries (geofences) and test whether the app correctly triggers events—like push notifications or security alerts—when the device enters or exits these predefined areas.
   Source: Windows Blog

5. **Network Condition Simulation:**
   Evaluate the app's location responsiveness across different network types (Wi-Fi, 3G, 4G, 5G) and in "Airplane Mode" to ensure it handles signal drops or slow data fetching gracefully.

   Source: YouTube (Test Scenarios)

6. **Approximate vs. Precise Permission Testing:**
   Test the app's behavior when a user grants only "approximate" location access (common in modern iOS and Android versions). The app must still provide relevant, though less granular, functionality.

   Source: Android Developers

7. **Battery Consumption Monitoring:**
   Geolocation services are power-intensive. Testing must include measuring the app's impact on battery life while the GPS is active, especially for background tracking features.

   Source: ResearchGate

8. **Geoblocking and Access Restriction:**
   Confirm that restricted content or services are correctly blocked or filtered when the device is detected in an unauthorized region, such as licensing restrictions for streaming apps.

   Source: TestMu AI

9. **Moving Patterns and Path Testing:**
   Simulate or physically perform movements at different speeds (walking vs. driving) to verify that real-time tracking, speed calculations, and route updates remain consistent and smooth.

   Source: Ministry of Testing

10. Automated Geolocation Scripts:

Integrate location parameters into automated testing frameworks like Appium or Selenium to perform repetitive regression testing of location-specific UI logic at scale.

Source: Perfecto

References

- How to Test Location-based Mobile Apps - ResearchGate
- Test your app's location workflows - Android Developers
- Mobile App Geolocation Testing: Challenges and Best Practices - Momentum Suite
- Power of Testing Websites and Mobile Apps Around The World - BrowserStack
- Testing and debugging your geofencing apps - Windows Blog
- How to test location based applications? | Test scenarios - YouTube
- Set mock location for Testing of Geolocation based Mobile Apps - Pcloudy
- How To Test Geolocation Of Websites And Mobile Apps - TestMu AI
- How do I test location based application? - Ministry of Testing
- A Guide to Geolocation Testing - Perfecto

END